

Split Inference on a Heterogeneous ESP32 Fleet

Project Work in Internet of Things

Giacomo Antonelli

September 2026

Abstract

Split inference runs the first k layers of a neural network on a microcontroller and sends the intermediate activation tensor to a server, which runs the rest. We built the whole pipeline on three ESP32-class boards of different power and measured what the choice of k costs. Over 3,813 requests with no corrupted deliveries, sending everything to the server was faster than every split point on every board: $5.1\times$ faster on an ESP32-S3 and $15.5\times$ faster on an ESP32-CAM. The weakest board could not host a split point at all. Cutting deeper made things worse, not better: the deeper cut shrank the tensor from 4,668 B to 888 B, but it saved only 2.7 ms of transfer while adding 26–164 ms of inference on the device. The per-stage breakdown explains why. Compute on the device is two orders of magnitude larger than anything else, while transfer, the very thing split inference is meant to reduce, takes 7–12 ms no matter which cut produced it. A sensitivity analysis over the measured stages puts break-even at a server $51\times$ slower than ours.

1 The question

For a network layer k , a split deployment pays three costs: running the first k layers on the device, sending the resulting activation tensor, and running the rest on the server. Cutting later means more work on the device and usually fewer bytes on the air. Split inference assumes that some middle value of k beats both extremes, and that the best k depends on the device.

This report tests that assumption by measuring it. It does not learn or adapt the split point: every policy here is a fixed rule, and the exhaustive sweep is the baseline an adaptive orchestrator would have to beat.

2 System

2.1 Hardware

Node	Tier	Board	Memory
11	A	ESP32-S3-WROOM-1 N16R8	8 MB octal PSRAM @ 80 MHz, vector ISA
21	B	ESP32-CAM (ESP32-D0WD)	8 MB quad PSRAM @ 80 MHz, no vector ISA
31	C	ESP32-D0WD-V3, plain	no PSRAM, ~ 150 KB usable SRAM
x86 laptop (Core 7 150U)			soft-AP, orchestrator, server tail

Table 1: The fleet. All three MCUs run at 240 MHz.

All boards run ESP-IDF v5.5.3 with TensorFlow Lite Micro. The server is the same machine that builds the firmware: it runs the Wi-Fi access point (2.4 GHz, because ESP32 radios have no 5 GHz band), the orchestrator, and the second half of the inference. Its CPU is capped to 400 MHz and the process is pinned to four cores. Section 5 explains what that costs the conclusions.

2.2 Model and split points

MobileNetV2, width multiplier 0.35 (every layer keeps 35% of the channels of the reference network, which is what makes it small enough for an MCU), $96 \times 96 \times 3$ input, full-int8 post-training quantization, two classes. Heads and tails are cut from one set of weights at the flatbuffer level (the `.tflite` file format itself, so operators and weights are copied byte-for-byte instead of being retrained or re-quantized), and the quantization parameters of the boundary tensor are copied unchanged from the head output to the tail input. A test checks that the int8 logits of $\text{full}(x)$ and $\text{tail}_k(\text{head}_k(x))$ are bit-identical over 200 random int8 inputs at every cut, so a split deployment provably returns the same prediction as full offload. Accuracy is therefore the same by construction, and latency is the only thing that differs between policies.

Three split points: **k0** (no split: the JPEG goes to the server, which runs the whole model), **k_shallow** (after inverted-residual block 2, boundary tensor $24 \times 24 \times 8 = 4,608$ B), and **k_deep** (after block 6, $6 \times 6 \times 24 = 864$ B).

2.3 Transport and measurement

UDP with fragmentation at the application level, CRC32 per tensor, NACK-based selective repeat over three rounds, and an abort path. One asyncio process on the server sends at most one request per node at a time. Every request writes one SQLite row that splits end-to-end latency into five stages that do not overlap and add up exactly to the total: **device** (flash read plus head inference), **uplink**, **queue**, **server**, **residual**. Every number in this report comes from a query over those rows.

3 Method

There are two kinds of run. The *sweep* measures every feasible $(\text{node}, \text{cut})$ pair, 200 requests each, one node at a time. Running one node at a time is what makes the numbers describe the cut itself rather than whatever the other nodes happened to be doing. The *policy runs* then test four fixed rules with the whole fleet active: **k0**, **k_shallow**, **k_deep**, and **best**, which replays the per-node winner from the sweep.

Every board reads the same 50 images from flash, in the same order, so all policies see the same inputs.

Seven gates guard the procedure, and each one invalidates everything after it if it fails. They cover split correctness, ARQ integrity under injected loss, verification of the whole harness against simulated nodes, arena feasibility per tier, fleet stability, sample counts, and dispatch correctness.

4 Results

4.1 Which split points each board can host

A board can only host a head if it can fit that head’s *arena*: the single block of RAM that TensorFlow Lite Micro reserves at startup and reuses for every intermediate tensor of a model. It is allocated once and never grows, so the size it needs is known before the model runs, and a board with less free RAM than that simply cannot run the head.

There are two results here. First, the weakest board cannot split, so on that hardware the only available policy is full offload. Second, the same cut needs a different amount of arena on different silicon: 149,972 B on the S3 against 142,296 B on the ESP32, for the same weights and tensor shapes, because the S3 build uses vector kernels that allocate their own scratch memory.

Tier	Arena source	<code>k_shallow</code>	<code>k_deep</code>	Feasible cuts
A	8 MB octal PSRAM	149,972 B	158,120 B	<code>k0</code> , <code>k_shallow</code> , <code>k_deep</code>
B	8 MB quad PSRAM	142,296 B	150,396 B	<code>k0</code> , <code>k_shallow</code> , <code>k_deep</code>
C	100 KB internal	needs 138,240 B, has 98,480 B		<code>k0</code> only

Table 2: Arena needed per head, measured on the device at boot. Tier C is short by 39 KB and cannot host any head: its firmware links none.

4.2 Bytes on the air

Sending the image costs fewer bytes than sending the tensor from the shallow cut, which is not what intuition suggests. The reason is that the image is compressed and the tensors are not. The raw $96 \times 96 \times 3$ input is 27,648 B, and JPEG takes it down to 1,775 B; the activation tensor at `k_shallow` is 4,608 B of uncompressed int8, six times smaller than the raw input but still $2.6\times$ larger than the JPEG. Only once the network has downsampled a few more times does the activation drop below the compressed image, which is what happens at `k_deep`.

Cut	Activation bytes	Mean bytes on air	Fragments	Retransmits/req
<code>k0</code>	1,775 (JPEG)	1,811	1.98	0.000
<code>k_shallow</code>	4,608	4,668	4.00	0.000
<code>k_deep</code>	864	888	1.00	0.000

Table 3: Data inflation. The shallow cut puts $2.6\times$ more on the air than just sending the JPEG; only the deep cut sends less.

4.3 Latency per node and split point

Node	Cut	n	Mean (ms)	p50 (ms)	p95 (ms)
11 (A)	<code>k0</code>	200	32.6	29.8	55.5
11 (A)	<code>k_shallow</code>	200	166.8	167.5	184.2
11 (A)	<code>k_deep</code>	200	190.0	189.9	209.6
21 (B)	<code>k0</code>	200	36.0	33.0	63.8
21 (B)	<code>k_shallow</code>	200	557.8	557.7	581.4
21 (B)	<code>k_deep</code>	200	719.9	720.3	743.6
31 (C)	<code>k0</code>	200	41.5	40.4	68.4

Table 4: The sweep: every feasible pair, measured on its own. No failures.

Full offload wins on every node, both at the mean and at p95, by $5.1\times$ on tier A and $15.5\times$ on tier B. And `k_deep` is worse than `k_shallow` on both capable tiers, even though it puts a fifth as much data on the air.

4.4 Where the time goes

The `device` stage dominates every split cut, and nothing else comes within two orders of magnitude. At the same time `uplink` barely changes across cuts, from 7.0 to 11.9 ms, even though the payload is $5\times$ larger for `k_shallow` than for `k_deep`. At these sizes airtime is driven by per-packet overhead rather than by bytes, so the data inflation in Table 3 is real but costs almost no time.

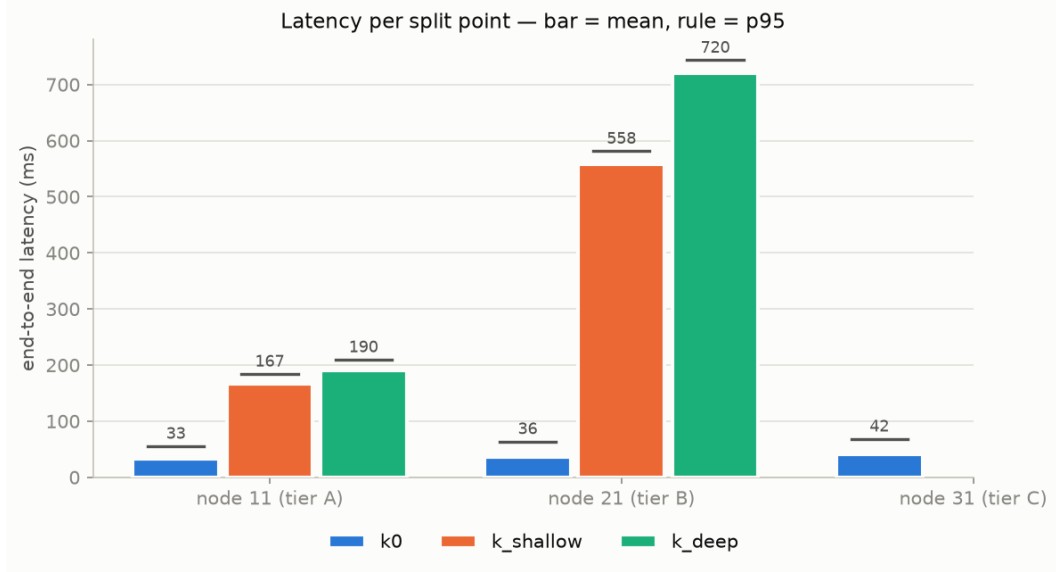


Figure 1: Latency per split point; the bar is the mean, the rule is p95.

Node	Cut	device	uplink	queue	server	residual
11 (A)	k0	19.4	7.6	0.1	4.3	1.1
11 (A)	k_shallow	154.5	9.7	0.1	1.5	1.1
11 (A)	k_deep	180.6	7.0	0.1	1.1	1.2
21 (B)	k0	22.3	8.5	0.1	4.1	1.1
21 (B)	k_shallow	543.9	11.0	0.1	1.6	1.2
21 (B)	k_deep	708.1	9.1	0.1	1.3	1.3
31 (C)	k0	24.3	11.9	0.1	4.2	1.1

Table 5: Mean stage breakdown in ms. The stages add up to the total.

4.5 The cost of running the fleet together

Policy	Node	Isolated (ms)	Concurrent (ms)	Difference
k0	11 (A)	32.6	42.4	+30.2%
best = k0	11 (A)	32.6	45.8	+40.7%
k_shallow	11 (A)	166.8	174.1	+4.4%
k_deep	11 (A)	190.0	193.5	+1.8%

Table 6: The same node running the same cut, measured alone and then with the whole fleet active. Policies that rely on the shared server pay about ten times more for concurrency than policies that compute on the device.

The three nodes share one single-worker server, so a policy with expensive server work makes every other node wait. The effect is there and goes in the expected direction: splitting really does protect a node from server contention. It does not change the ranking, though, since k0 at 42.4 ms with its penalty is still $4.1\times$ faster than k_shallow at 174.1 ms.

4.6 Reliability

After fixing some protocol issues, the system became more stable.

Run	Requests	OK	TIMEOUT	CRC	LOST	Reboots
sweep	1,400	1,400	0	0	0	0
k0	603	600	0	0	3	0
k_shallow	610	600	9	0	1	0
k_deep	600	600	0	0	0	0
best	600	600	0	0	0	0

Table 7: No corrupted deliveries across 3,813 requests, and no node rebooted during any run. The worst failure rate is 1.6%.

5 Threats to validity

The server is an x86 laptop, not an MCU-class edge box, and even capping the CPU to 400 MHz and limiting the cores, kept the results very cler towards one direction. For a better comparison, more edge devices should be used, such that more workload is put on the server, making local inference more competitive.

The input is done over stored images, not live capture. In a real scenario, the edge should also take in account for the reading and processing time of the input data.

6 Conclusion

No split point beat full offload on any board, cutting deeper made things worse, and the weakest board could not split at all. Inference on the device dominates the budget by two orders of magnitude, while transfer stays roughly constant across cuts, so the parameter this project set out to tune has almost no effect. The negative result is quantified rather than just asserted: the server would have to be $51\times$ slower before the shallow cut broke even.